



AULA 04

SISTEMA DE VENDAS E GESTÃO ONLINE



Parte 1

Introdução

{JORNADA}
PYTHON
100% GRATUITO E ONLINE

O que Vamos Aprender?

Na aula Sistema de Vendas com Python nós vamos criar um sistema completo para cadastrar e analisar vendas.

Imagine que você trabalha em uma empresa que tem vendedores. Toda vez que uma venda acontece, alguém precisa registrar as informações: a data, quem vendeu, o que vendeu, a quantidade e o valor. Hoje isso pode estar sendo feito em papel, em e-mail ou numa planilha que fica bagunçada.

Com esse projeto em Python, qualquer pessoa na empresa poderá:

- **Cadastrar uma venda** em segundos usando um formulário simples
- **Visualizar todas as vendas** em uma tabela organizada e interativa
- **Acompanhar o faturamento e desempenho** dos vendedores e produtos no dashboard

E o melhor: o sistema vai funcionar como um site, que qualquer pessoa pode acessar pelo navegador — sem precisar instalar nada de especial!

Para isso, vamos usar três bibliotecas do Python: o Streamlit para criar a interface do sistema, o Pandas para manipular os dados e o Plotly para criar os gráficos. Não se preocupe com elas agora, vamos explicar cada uma nos próximos slides.

Veja como vai ficar o resultado final do nosso projeto! Um sistema completo com formulário de cadastro na sidebar, tabela de vendas e dashboard com gráficos e métricas de faturamento.



Parte 2

Bibliotecas Necessárias

Instalação das Bibliotecas

Para essa aula nós vamos utilizar três bibliotecas do Python. Uma biblioteca é como uma "caixa de ferramentas" com funções prontas que outras pessoas programaram para você usar.

Antes de instalar, entenda o que cada biblioteca faz:

Streamlit — É a ferramenta que transforma seu código Python em um sistema web com interface visual. Com ela, criamos botões, formulários, tabelas e gráficos sem precisar saber nada de HTML, CSS ou JavaScript. É como ter um construtor de sistemas completo dentro do Python.

Pandas — Trabalha com dados em formato de tabela, como uma planilha do Excel dentro do Python. Com ela podemos ler arquivos de dados, editar informações, fazer cálculos e organizar tudo em tabelas.

Plotly — Cria gráficos interativos e profissionais com poucas linhas de código. Diferente de gráficos comuns (imagens estáticas), os do Plotly permitem que o usuário passe o mouse para ver os valores e explore os dados visualmente.

Para instalar todas de uma vez, execute no terminal:
`pip install streamlit pandas plotly`

Ou instale individualmente:

Streamlit – `pip install streamlit`

Pandas – `pip install pandas`

Plotly – `pip install plotly`

Você precisa das bibliotecas instaladas para que consiga fazer esse projeto. Caso contrário vai ter mensagens de erro ao executar o código.

Documentações para consulta:

Streamlit - <https://docs.streamlit.io/>

Pandas - <https://pandas.pydata.org/docs/>

Plotly - <https://plotly.com/python/>

Obs.: Lembrando que você já vai ter disponível o arquivo `requirements.txt` para saber o que precisa ser instalado.

Parte 3

Criando a Tela do Sistema

Passo 1: Criando a Tela do Sistema

Agora vamos começar a criar o nosso sistema! O primeiro passo é importar as bibliotecas e criar a tela principal.

O que significa "import"?

Quando usamos import no Python, estamos dizendo ao programa: "carregue essa biblioteca para que eu possa usar suas funções". Pense como abrir um aplicativo antes de começar a trabalhar — você não consegue usar o Excel sem abri-lo primeiro.

```
import streamlit as st
import pandas as pd
import plotly.express as px
```

O que é o "as st", "as pd", "as px"?

São apelidos (atalhos). Com eles, em vez de escrever `streamlit.write(...)`, você escreve apenas `st.write(...)`. Muito mais prático! É como salvar um contato no celular: em vez de discar o número inteiro, você usa o nome.

Criando o título do sistema:

```
st.write('# Sistema de Vendas')
```

O símbolo # antes do texto faz o Streamlit exibir o título em tamanho grande. Use ## para título médio e #### para título pequeno.

Carregando os dados do arquivo:

```
tabela = pd.read_csv('vendas.csv')
```

O arquivo vendas.csv é como uma planilha salva em formato texto — cada linha é uma venda e as colunas são separadas por vírgula. O `pd.read_csv` lê esse arquivo e o transforma em uma tabela que o Python consegue manipular.

Para executar o sistema, abra o terminal e execute:

```
streamlit run main.py
```

```
# pip install streamlit pandas plotly
import streamlit as st
import pandas as pd
import plotly.express as px

# Passo 1: Criar a tela do sistema
st.write("# Sistema de Vendas")

tabela = pd.read_csv("vendas.csv")
```

Parte 4

Criando o Formulário de Cadastro

Passo 2: Criando o Formulário de Cadastro

Com a tela criada, agora vamos adicionar o formulário de cadastro na barra lateral (sidebar) do sistema.

O que é a sidebar?

A sidebar é a barra lateral que aparece no lado esquerdo da tela. É o espaço ideal para formulários porque fica separado da área principal dos dados, sem atrapalhar a visualização.

Para colocar elementos na sidebar, escrevemos `st.sidebar.` antes de cada comando. Veja cada campo:

`st.sidebar.write('## Cadastrar venda')`
Cria o título do formulário na sidebar.

`data = st.sidebar.date_input('Data')`
Campo de data com calendário interativo. O usuário clica e escolhe a data. O resultado fica salvo na variável **data**.

`vendedor = st.sidebar.selectbox('Vendedor', ['Ana', 'Bruno', 'Carla'])`
Cria uma lista suspensa (dropdown) com opções fixas para o usuário escolher. O **selectbox** é perfeito quando há um conjunto limitado de opções.

`produto = st.sidebar.selectbox('Produto', ['Notebook', 'Celular', 'Fone'])`

Outro dropdown, agora para o produto vendido. Funciona igual ao de vendedor.

`quantidade = st.sidebar.number_input('Quantidade', step=1)`
Campo numérico. O **step=1** faz o número aumentar e diminuir de 1 em 1 quando o usuário clica nas setas laterais.

`valor = st.sidebar.number_input('Valor')`
Campo numérico para o valor da venda. Aceita decimais, como 3500.50.

`botao = st.sidebar.button('Cadastrar venda')`
Cria o botão de cadastro. Quando clicado, a variável **botao** recebe o valor **True**. Quando não foi clicado, recebe **False**.

```
# Passo 2: Criar o formulário de cadastro
st.sidebar.write("## Cadastrar venda")
data = st.sidebar.date_input("Data")
vendedor = st.sidebar.selectbox("Vendedor", ["Ana", "Bruno", "Carla"])
produto = st.sidebar.selectbox("Produto", ["Notebook", "Celular", "Fone"])
quantidade = st.sidebar.number_input("Quantidade", step=1)
valor = st.sidebar.number_input("Valor")
botao = st.sidebar.button("Cadastrar venda")
```

Parte 5

Salvando e Mostrando as Vendas

Passo 3: Salvando a Venda na Base de Dados

Com o formulário criado, agora vamos programar o que acontece quando o usuário clica no botão de cadastrar.

Entendendo o "if" (SE condicional):

O if é um dos comandos mais importantes do Python. Em português significa "SE". Ele executa um bloco de código apenas SE uma condição for verdadeira. Nesse caso: SE o botão foi clicado (botao = True), execute as linhas abaixo.

if botao:

```
nova_venda = [data, vendedor, produto, quantidade, valor]
```

O que são os colchetes []?

Os colchetes criam uma lista — uma "caixinha" que guarda vários valores em sequência. Aqui estamos guardando em nova_venda todos os dados que o usuário preencheu no formulário.

```
tabela.loc[len(tabela)] = nova_venda
```

Adiciona a nova venda na próxima linha disponível da tabela. O **len(tabela)** retorna o número de linhas existentes, que é o índice da posição onde a nova linha deve entrar.

```
tabela.to_csv('vendas.csv', index=False)
```

Salva toda a tabela atualizada de volta no arquivo vendas.csv. O **index=False** evita que o Pandas adicione uma coluna extra de numeração no início de cada linha.

```
st.success('Venda cadastrada!')
```

Exibe uma caixa verde de sucesso na tela, confirmando ao usuário que a venda foi salva com sucesso.

Assim, cada vez que o usuário cadastrar uma venda, ela será salva permanentemente no arquivo CSV e ficará disponível para consulta mesmo após fechar e reabrir o sistema!

```
# Passo 3: Salvar a venda na base de dados
if botao:
    nova_venda = [data, vendedor, produto, quantidade, valor]
    tabela.loc[len(tabela)] = nova_venda
    tabela.to_csv("vendas.csv", index=False)
    st.success("Venda cadastrada!")
```

Venda cadastrada!

Passo 4: Mostrando a Base de Dados na Tela

O quarto passo é exibir todas as vendas cadastradas na tela principal do sistema, para que o usuário possa visualizar o histórico completo.

Para isso, usamos apenas dois comandos simples:

```
st.write('## Vendas cadastradas')
```

Cria um subtítulo na página. No Streamlit, o número de # define o tamanho do título:

→ título grande (como o "Sistema de Vendas" que criamos no Passo 1)

→ título médio

→ título pequeno

```
st.dataframe(tabela)
```

Exibe a variável **tabela** como uma tabela interativa na tela do sistema.

```
# Passo 4: Mostrar a base de dados na tela
st.write("## Vendas cadastradas")
st.dataframe(tabela)
```

O que é a variável "tabela"?

Lembra que no Passo 1 carregamos o arquivo com `pd.read_csv('vendas.csv')`? A variável `tabela` guarda toda a planilha. Quando o usuário cadastra uma nova venda (Passo 3), a `tabela` é atualizada, e o `st.dataframe` mostra essa versão mais recente automaticamente.

Com o `st.dataframe`, o usuário pode:

- Clicar nos cabeçalhos das colunas para ordenar os dados
- Rolar horizontalmente para ver todas as colunas
- Visualizar todas as vendas registradas no arquivo de forma clara e organizada

Vendas cadastradas

	data	vendedor	produto	quantidade	valor
111	2026-08-18	Carla	Notebook	1	3500
112	2026-08-19	Ana	Fone	2	300
113	2026-08-21	Ana	Celular	1	2000
114	2026-08-21	Ana	Notebook	3	10500
115	2026-08-22	Bruno	Notebook	2	7000
116	2026-08-23	Ana	Notebook	1	3500
117	2026-08-24	Carla	Celular	1	2000
118	2026-08-27	Ana	Celular	4	8000
119	2026-08-31	Bruno	Celular	3	6000
120	2026-09-10	Ana	Notebook	3	1500

Parte 6

Criando o Dashboard

Passo 5: Criando o Dashboard

O quinto e último passo é criar o dashboard com métricas e gráficos para visualizar o desempenho das vendas.

```
st.write('## Dashboard')
```

Cria o subtítulo da seção.

Calculando e exibindo o faturamento total:

```
soma = tabela['valor'].sum()
st.metric('Faturamento total', f'R${soma}')
```

Veja o que cada parte faz:

tabela['valor'] — Seleciona apenas a coluna "valor" da tabela. É como clicar numa coluna específica no Excel para trabalhar só com ela.

.sum() — Soma todos os números dessa coluna. Se a coluna tiver 1000, 2000 e 3500, o resultado será 6500.

f'R\${soma}' — Isso é um **f-string** (string formatada). O **f** antes das aspas permite colocar variáveis dentro do texto usando {}. O Python substitui {soma} pelo valor da variável automaticamente. Exemplo: 'R\$6500.0'.

st.metric — Cria um card visual destacado com um título acima e um valor grande abaixo. É perfeito para indicadores importantes como faturamento total, número de vendas, média por venda, etc.

```
# Passo 4: Mostrar a base de dados na tela
st.write("## Vendas cadastradas")
st.dataframe(tabela)

# Passo 5: Criar o dashboard
st.write("## Dashboard")
soma = tabela["valor"].sum()
st.metric("Faturamento total", f"R${soma}")

grafico = px.bar(tabela, x="vendedor", y="valor", color="produto")
st.plotly_chart(grafico)

grafico2 = px.pie(tabela, names="produto", values="valor")
st.plotly_chart(grafico2)
```

Passo 5: Gráficos de Vendas

Agora vamos adicionar os gráficos ao dashboard. Vamos criar dois gráficos diferentes usando o Plotly Express (o apelido `px` que importamos no Passo 1).

Gráfico de Barras — Vendas por Vendedor:

```
grafico = px.bar(tabela, x='vendedor', y='valor', color='produto')  
st.plotly_chart(grafico)
```

Entendendo os parâmetros do `px.bar`:

tabela → os dados que vamos usar para montar o gráfico

x='vendedor' → o eixo horizontal mostra os nomes dos vendedores

y='valor' → o eixo vertical mostra os valores das vendas

color='produto' → cada produto recebe uma cor diferente nas barras, criando um gráfico empilhado por produto

Gráfico de Pizza — Participação por Produto:

```
grafico2 = px.pie(tabela, names='produto', values='valor')  
st.plotly_chart(grafico2)
```

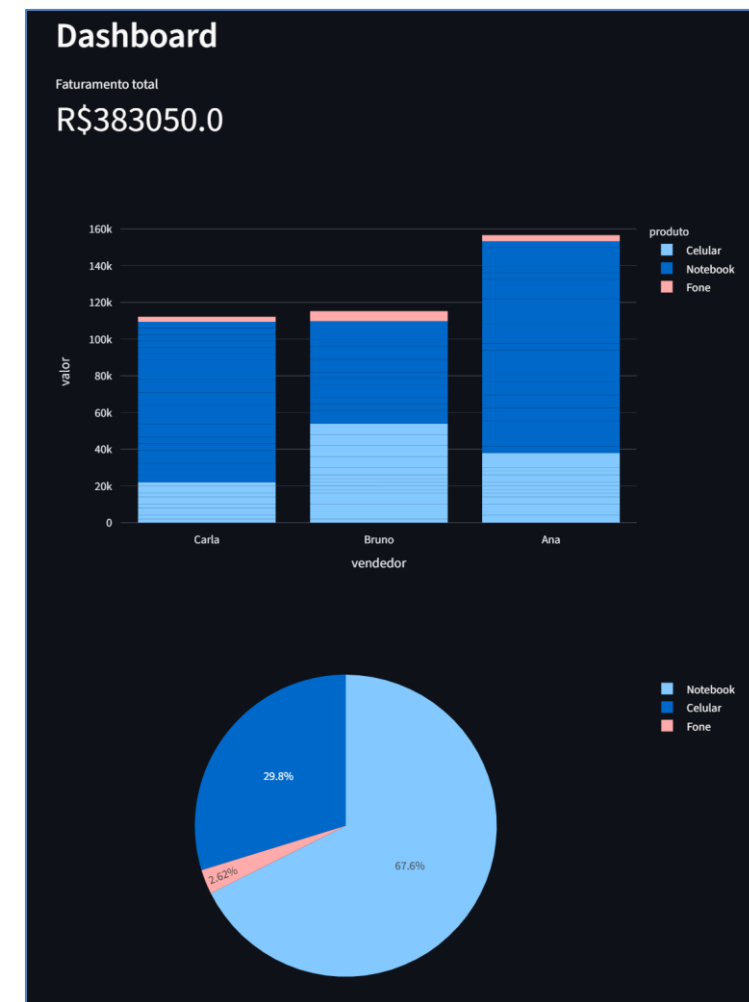
Entendendo os parâmetros do `px.pie`:

names='produto' → define os "pedaços" da pizza — um pedaço para cada produto

values='valor' → define o tamanho de cada pedaço conforme o faturamento do produto

st.plotly_chart — Exibe o gráfico interativo na tela do sistema.

Dica: Os gráficos do Plotly são interativos! Passe o mouse sobre barras ou fatias para ver os valores exatos. Clique na legenda para mostrar ou esconder produtos específicos.



Parte 7

Deploy do Sistema

Deploy do Sistema

Resumo do Código Completo

1. Criação da Interface do Sistema

A aplicação inicia exibindo o título “*Sistema de Vendas*”.

A base de dados é carregada a partir do arquivo **vendas.csv**, que contém o histórico de vendas.

A interface é dividida em:

- Área principal (exibe tabela e gráficos).
- Barra lateral (formulário de cadastro).

2. Formulário de Cadastro de Vendas

Localizado na barra lateral para não interferir na visualização dos dados.

Campos disponíveis:

- Data da venda** (date_input)
- Vendedor** (selectbox)
- Produto** (selectbox)
- Quantidade** (number_input)
- Valor da venda** (number_input)

Botão “**Cadastrar venda**” dispara o processo de gravação.

O formulário garante padronização dos dados e evita erros de digitação.

3. Salvamento da Venda na Base de Dados

Ao clicar no botão:

- Uma nova linha é criada com os dados informados.
- A linha é adicionada ao DataFrame existente.
- O arquivo **vendas.csv** é atualizado automaticamente.

Uma mensagem de sucesso confirma o cadastro.

O sistema funciona como um pequeno banco de dados local baseado em CSV.

4. Exibição das Vendas na Tela

A tabela completa de vendas é exibida na interface principal.

Permite ao usuário visualizar:

- Histórico completo
- Dados recém cadastrados

A tabela é interativa, permitindo rolagem e inspeção rápida.

5. Dashboard com Indicadores e Gráficos

Exibe o **faturamento total** somando a coluna *valor*.

Gráficos gerados com Plotly:

Gráfico de barras: compara o valor vendido por vendedor, segmentado por produto.

Gráfico de pizza: mostra a participação de cada produto no faturamento.

O dashboard facilita a análise visual e tomada de decisão.

```
# pip install streamlit pandas plotly
import streamlit as st
import pandas as pd
import plotly.express as px

# Passo 1: Criar a tela do sistema
st.write("# Sistema de Vendas")

tabela = pd.read_csv("vendas.csv")

# Passo 2: Criar o formulário de cadastro
st.sidebar.write("## Cadastrar venda")
data = st.sidebar.date_input("Data")
vendedor = st.sidebar.selectbox("Vendedor", ["Ana", "Bruno", "Carla"])
produto = st.sidebar.selectbox("Produto", ["Notebook", "Celular", "Fone"])
quantidade = st.sidebar.number_input("Quantidade", step=1)
valor = st.sidebar.number_input("Valor")
botao = st.sidebar.button("Cadastrar venda")
```

```
# Passo 3: Salvar a venda na base de dados
if botao:
    nova_venda = [data, vendedor, produto, quantidade, valor]
    tabela.loc[len(tabela)] = nova_venda
    tabela.to_csv("vendas.csv", index=False)
    st.success("Venda cadastrada!")

# Passo 4: Mostrar a base de dados na tela
st.write("## Vendas cadastradas")
st.dataframe(tabela)

# Passo 5: Criar o dashboard
st.write("## Dashboard")
soma = tabela["valor"].sum()
st.metric("Faturamento total", f"R${soma}")

grafico = px.bar(tabela, x="vendedor", y="valor", color="produto")
st.plotly_chart(grafico)

grafico2 = px.pie(tabela, names="produto", values="valor")
st.plotly_chart(grafico2)
```

Passo 1: Subindo os Arquivos no GitHub

Agora que o sistema está pronto, vamos publicá-lo na internet! Para isso usamos duas ferramentas gratuitas: o GitHub guarda os arquivos e o Streamlit Cloud publica o sistema online.

O que é deploy?

Deploy é o ato de publicar seu sistema para que outras pessoas possam acessá-lo pelo navegador, de qualquer lugar do mundo.

Passo 1 — Crie uma conta no GitHub

Acesse www.github.com e crie uma conta gratuita. O GitHub é uma plataforma para guardar e compartilhar código na internet.

Passo 2 — Crie um novo repositório

Clique em "New repository". Dê um nome (ex: sistema-de-vendas), deixe como "Public" e clique em "Create repository". Um repositório é como uma pasta online para seu projeto.

Passo 3 — Faça upload dos 3 arquivos

Na próxima tela, clique em "uploading an existing file" e envie os arquivos:

- **main.py** — o código do sistema
- **requirements.txt** — lista de bibliotecas necessárias
- **vendas.csv** — a base de dados inicial

Passo 4 — Confirme o envio

Clique em "Commit changes" para salvar os arquivos. Pronto! Os arquivos estão no GitHub e prontos para o próximo passo.

The screenshot shows the 'Create a new repository' page on GitHub. It has two sections: 'General' and 'Configuration'. In the 'General' section, the 'Owner' is 'heitorcatunda' and the 'Repository name' is 'sistema-de-vendas'. A green checkmark indicates the name is available. The 'Description' is 'Sistema de vendas'. In the 'Configuration' section, 'Choose visibility' is set to 'Public', 'Add README' is 'Off', 'Add .gitignore' is 'No .gitignore', and 'Add license' is 'No license'.

The screenshot shows the 'Commit changes' dialog box. It has a title bar with a close button. Below the title, there is a list of files: 'main.py', 'requirements.txt', and 'vendas.csv'. The 'Commit changes' section has a text input for 'Add files via upload' and a larger text area for 'Add an optional extended description...'. At the bottom, there are two buttons: 'Commit changes' (green) and 'Cancel' (gray).

Passo 2: Publicando no Streamlit Cloud

Com os arquivos no GitHub, vamos agora conectá-los ao Streamlit Cloud para publicar o sistema online.

Passo 1 — Acesse o Streamlit Cloud

Vá em share.streamlit.io e faça login usando a mesma conta do GitHub. O Streamlit vai reconhecer seus repositórios automaticamente.

Passo 2 — Crie um novo app

Clique em "New app" e preencha as informações do projeto:

- **Repository:** selecione o repositório criado (ex: sistema-de-vendas)
- **Branch:** main
- **Main file path:** main.py

Passo 3 — Publique o sistema

Clique em "Deploy!" e aguarde alguns minutos. O Streamlit vai ler seus arquivos do GitHub, instalar as bibliotecas do requirements.txt e publicar o sistema. Simples assim!

Passo 4 — Compartilhe o link

Ao finalizar, você receberá um link público (ex: seusuuario.streamlit.app/sistema-de-vendas). Qualquer pessoa pode acessar pelo celular ou computador sem precisar instalar nada!

Dica: Sempre que você atualizar os arquivos no GitHub, o Streamlit Cloud atualiza o sistema online automaticamente.

Veja o projeto desta aula publicado em:

<https://sistemadevendasteste-lira.streamlit.app/>

The image shows the Streamlit Cloud deployment process and the resulting application. On the left, the 'Deploy an app' form is filled out with the repository 'heitorcatunda/sistema-de-vendas', branch 'main', and main file path 'main.py'. The 'Deploy' button is visible. On the right, the deployed application is shown. It has a dark theme and includes a 'Cadastrar venda' form on the left and a 'Sistema de Vendas' dashboard on the right. The dashboard displays a table of registered sales and a total revenue summary.

Cadastrar venda

Data: 2026/09/16

Vendedor: Ana

Produto: Notebook

Quantidade: 0

Valor: 0,00

Sistema de Vendas

Vendas cadastradas

	data	vendedor	produto	quantidade	valor
0	2026-06-03	Carla	Celular	1	2000
1	2026-06-03	Carla	Celular	1	2000
2	2026-06-04	Bruno	Notebook	2	7000
3	2026-06-04	Carla	Celular	1	2000
4	2026-06-04	Carla	Notebook	1	3500
5	2026-06-06	Carla	Notebook	1	3500
6	2026-06-06	Ana	Notebook	1	3500
7	2026-06-08	Ana	Fone	1	150
8	2026-06-08	Ana	Fone	2	300
9	2026-06-08	Carla	Fone	2	300

Dashboard

Faturamento total

R\$383050.0

Conclusão

Conclusão

A ideia desse projeto foi mostrar como o Python pode criar soluções reais e completas de forma surpreendentemente simples. Com menos de 50 linhas de código, criamos um sistema que:

- Tem uma interface web profissional e interativa que funciona no navegador
- Permite cadastrar vendas com um formulário fácil de usar
- Salva as informações permanentemente em uma base de dados (arquivo CSV)
- Exibe todas as vendas em uma tabela organizada e interativa
- Apresenta um dashboard com métricas de faturamento e gráficos de análise

Agora imagine aplicar isso no dia a dia de uma empresa! Qualquer funcionário poderia cadastrar uma venda em segundos, e o gerente veria o dashboard em tempo real para tomar decisões baseadas em dados reais.

O link do projeto online é: <https://sistemadevendasteste-lira.streamlit.app/>

Desafio para você:

Tente reproduzir o projeto do zero e depois personalize com seus próprios dados, vendedores e produtos. Algumas ideias para evoluir o sistema:

- Adicionar novos campos como "Região" ou "Canal de Venda"
- Criar filtros para ver vendas de um período específico
- Adicionar mais gráficos com diferentes análises
- Mudar as cores e o visual do sistema

Você acabou de criar um projeto real com Python, e isso é incrível! Espero que tenha gostado da aula e vejo você nos próximos projetos!

{JORNADA} PYTHON

100% GRATUITO E ONLINE

Ainda não segue a gente no Instagram e nem é inscrito no nosso canal do Youtube? Então corre lá!



@hashtagprogramacao



youtube.com/hashtag-programacao

